



Three-User Self-Drive Car Rental System (Customers, Owners, Admin)

In this enhanced design, the system supports **three user roles**: **Customers** who rent cars, **Owners** who list their cars for rent, and an **Admin** who manages the platform. For example, one project description explicitly notes: *"There are 3 main users in the application: the car owner, the admin, and the customer. The admin will act as a middle man between the car owner and the customer"* ¹. We will outline the architecture, modules, data flow, and database updates to accommodate these roles.

System Architecture

We maintain a **three-tier architecture** (frontend, backend, database), but now with separate dashboards/views for each role. The React frontend will render different pages/components depending on user type (customer, owner, or admin). The Spring Boot server will have controllers and services for owner-related actions as well as customer ones. Owners, like customers, use the web/mobile app, so they interact with the same server and database. We still use MySQL as the data tier. Within Spring Boot, the layered structure remains: controllers → services → repositories ² ³. An Owner's actions (e.g. listing a car) flow through these layers just as Customer actions do. All three user types are authenticated via the same **Auth** system, but with role-based access control (e.g. using Spring Security).

Major Modules and Functionalities

We update the previous modules and add owner-specific ones:

- **Authentication Module.** Handles **Registration/Login** for all user types. We will have separate registration flows or account types (Customer vs. Owner), though technically they can share a **Users** table with a **role** field. An **Admin** account is managed by system admins and may not self-register. For example, each user registers with email/password and selects role "Owner" or "Customer" (or an admin adds their account). The database **Users** table might include **role** (e.g. "CUSTOMER" or "OWNER"). Authentication endpoints (`/api/auth/register`, `/api/auth/login`) will accept user type. Only **Owners** can access owner-specific pages; this is enforced in the backend (e.g. `@PreAuthorize("hasRole('OWNER')")`) and in the frontend routing.
- **Owner Management Module. Purpose:** Allows car owners to onboard and manage their fleet.
Functionality:
 - **Owner Registration/Login:** Owners sign up (separate from customers) and log in.
 - **Owner Dashboard:** After login, owners see a dashboard showing their listed cars and rental requests.
 - **Car Listing:** Owners create and update **Vehicle Listings** with details (make, model, year, pricing, availability schedule, images, etc.). Owners specify when and where each car is available. According

to a peer-to-peer rental overview, “Car owners can create detailed vehicle listings, including [info like] availability” ⁴ .

- **Booking Requests:** Owners view incoming booking requests for their cars and **accept or decline** them ⁵ . For example, the system notifies the owner when a customer requests a booking; the owner can approve it (setting booking status to “Confirmed”) or reject it.
- **Communication:** Owners can communicate with customers about bookings (e.g. confirming times) using in-app chat/messaging. In such platforms, “communication channels [allow] car owners and renters to exchange messages... and reminders” ⁶ .
- **Earnings and Payouts:** Owners track earnings on completed rentals. Payments from customers are held by the platform/admin, then transferred to owners (minus commission). One project notes: “*Will get money from Customer and transfer to Car Owner after keeping commission*” ⁷ . We may implement a payout service where the admin settles with owners monthly.
- **Ratings and Reviews:** Owners receive ratings from customers after rentals and can also review renters. Ratings build trust in the marketplace ⁸ .

React Pages/Components: e.g. `OwnerRegisterPage`, `OwnerLoginPage`, `OwnerDashboard`, `MyCarsPage`, `OwnerBookingsPage`, `OwnerPayoutsPage`. Components for adding/editing a car (`<CarForm>`), listing owner’s cars, and viewing requests. These pages call new APIs (below).

Backend APIs: - `POST /api/owners/register`, `POST /api/owners/login` - owner auth.

- `GET /api/owners/{ownerId}/cars` - list owner’s cars.
- `POST /api/owners/{ownerId}/cars` - add new car (fields: category, price, location, etc.).
- `PUT /api/owners/{ownerId}/cars/{carId}` - update car details or availability.
- `GET /api/owners/{ownerId}/bookings` - view bookings (requests) for owner’s cars.
- `POST /api/owners/{ownerId}/bookings/{bookingId}/approve` - owner approves a booking request (sets status to Confirmed) ⁵ .
- `POST /api/owners/{ownerId}/bookings/{bookingId}/reject` - owner rejects a request.
- `GET /api/owners/{ownerId}/earnings` - view completed bookings and earnings.
- (Any other needed CRUD endpoints for owner profile, etc.)

- **Car Listing/Search Module (updated).** Similar to before, but now *only Owners* add cars. The search for customers remains: customers search by location, date, category. The backend ensures only cars marked available by owners appear. A noteworthy point from peer-to-peer systems: renters can filter by price, reviews, capacity, etc. (The GitHub user features mention filtering by price, reviews, seating, model ⁹ , but we focus on core tech). Search API (`GET /api/cars`) now returns cars from all owners (maybe limited to verified owners).

- **Booking Module (updated).** Flow: A **Customer** browses and requests a booking on an owner’s listed car. The request (`POST /api/bookings`) creates a booking record with status “PendingOwnerApproval”. The system notifies the car’s Owner and the Admin. The Owner then **approves or rejects** the booking (via the endpoint above). Once **approved by the Owner**, the customer proceeds to pay. This two-step process is common in P2P rental platforms. After payment, the booking status becomes “Confirmed”. (In some designs, admin might also review/verify before final confirmation.) The Booking table in the database will include `owner_id` (or derive via `car_id` → Car→Owner) and a `status` field (Pending, Approved, Confirmed, Completed). Our earlier booking fields still apply (from/to dates, pickup/drop locations, etc.), but with an added approval step.

- **Payment & Payout Module.** When a customer completes payment (`POST /api/payments`), the funds go to the platform (admin account). The system calculates the owner's share (e.g. 80%) and the admin's commission (e.g. 20%). The GitHub example says the admin **transfers payment to owner after commission deduction** ⁷ . We might create a `Transactions` or `Payouts` table that logs payouts to owners. For customers, existing payment endpoints apply, but after payment we also record `ownerShare = paymentAmount * 0.8` and schedule a payout. On the frontend, customers use their normal Payment page. Owners might have a **Payouts** page showing upcoming and completed payouts. Admin has an interface to view all transactions and initiate owner payments.
- **Chat/Communication Module (updated).** Expand the chat so that **any two parties** (owner-customer, owner-admin, or admin-customer) can communicate. In practice, we implement it so owners and customers can chat directly (e.g. Customer asks Owner about car). Admins can step in if needed. We reuse the WebSocket/STOMP approach. For example, one source emphasizes that communication is a key feature: *"The software provides communication channels for car owners and renters to exchange messages... and reminders"* ⁶ . We ensure our chat UI and backend allow selecting the recipient (owner or customer). Message storage (`ChatMessage` table) will include `sender_id` and `recipient_id` as before ¹⁰ .
- **Admin Module (updated).** Admin functionality broadens:
 - **Admin Verification:** Admin can approve or suspend Owner accounts (verify documents).
 - **Platform Management:** Admin views all users and owners, all listings and bookings.
 - **Commission Settings:** Admin configures commission rates, taxes.
 - **Payouts:** Admin processes payouts to owners and charges customers.
 - **Content Control:** Admin can remove inappropriate listings or block users.
 - **Support/Chat:** Admin monitors chats and can intervene.

The GitHub description states: *"Will implement the car owner's vehicle conditions... Will act as intermediary between Car Owner and User... transfer [money] after commission"* ¹¹ . Admin pages in React might include `AdminDashboard` , `ManageOwners` , `ManageCustomers` , `ManageBookings` , and `PlatformSettings` . Corresponding APIs allow admin actions (`/api/admin/*`).

Frontend (React) Changes

We will add owner-specific pages and UI components, while adjusting existing ones:

- **Owner Pages:** Components such as `OwnerRegisterPage` , `OwnerLoginPage` , `OwnerDashboard` , `MyCarsPage` (showing owner's listed cars), `OwnerCarForm` (add/edit car), `OwnerBookingsPage` (view booking requests for their cars), and `OwnerEarningsPage` . The Owner dashboard can use Bootstrap cards or tables. We will also adjust navigation menus based on role (owners see "My Cars", "My Bookings", customers see "Book a Car", admin sees "Admin Panel", etc.).

- **Customer Pages:** Largely unchanged, but now customer booking flows must include an approval step. We may show a “Pending Approval” status on bookings. After booking, if waiting for owner, show appropriate message.
- **Shared Components:** Some components (Chat, Feedback, CarList, CarDetails) serve all roles with minor tweaks. For instance, on the car detail page, owners might see a special “Edit Listing” button, while customers see “Request Booking”.
- **Role-Based Routing:** Using React Router (or similar) we ensure only authorized roles see certain routes. For example, routes under `/owner/*` load owner components, under `/admin/*` load admin pages.

No embedded images are needed for this explanation, but we will cite sources illustrating multi-party flows.

Backend (Spring Boot) Updates

- **New Entities:** Add an `Owner` entity (and table) to store owner-specific data (ID, user reference, payout info, etc.). Alternatively, use the existing `User` table with a `role="OWNER"` and have an `OwnerProfile` table linked to it. The `Car` entity now has a foreign key to `Owner` (e.g. `owner_id`). Bookings link indirectly to Owner via the car.
- **Controllers & Services:**
 - **OwnerController:** Handles owner sign-up (`/owners/register`), and all owner actions (CRUD cars, view bookings, approve/reject).
 - **BookingController:** Extended so that it checks owner approval. E.g. in `createBooking()`, set status to “PendingOwner”. A new endpoint `approveBooking(bookingId)` checks the user is the car’s owner before confirming.
 - **PaymentController:** After customer payment, create transactions for owner payouts.
 - **AdminController:** For admin management of owners, commission, etc.
- **Security/Permissions:** Spring Security is configured so that endpoints starting with `/owners/**` require owner role, `/customers/**` require customer role, and `/admin/**` require admin role. The earlier approach (no JWT, session-based) still applies, but ensure role checks.
- **API Endpoints (examples):**
 - `POST /api/owners/register` – create owner account (verified by admin later).
 - `GET /api/owners/{id}/cars` – owner’s car list.
 - `PUT /api/owners/{id}/cars/{carId}` – modify a car.
 - `POST /api/bookings` – customer requests booking.
 - `POST /api/bookings/{id}/approve` – owner approves (see above).
 - `POST /api/admin/payouts` – admin processes owner payout.

Database Design (MySQL)

We revise the schema to include owners:

- **Users Table:** Contains all user accounts (customers and owners) with fields `user_id` PK, `name`, `email`, `password_hash`, `role`. `role` is "CUSTOMER" or "OWNER" (or "ADMIN").
- **Owners Table (optional):** If needed for extra fields, an `owners` table can store `owner_id` PK, `user_id` FK→Users, `bank_account`, `verified` (boolean), `rating`, etc..
- **Cars Table:** Now has `owner_id` FK referencing the owning user. Fields: `car_id`, `owner_id` FK, `model`, `make`, `year`, `category_id`, `location`, `price_per_day`, `availability_status`, `image_url`, The owner sets these.
- **Bookings Table:** (`booking_id`, `car_id` FK→Cars, `user_id` FK→Users, `from_date`, `to_date`, `pickup_loc`, `dropoff_loc`, `status`). We add `status` values to include "PendingOwner" or similar. It automatically links to the owner through `car_id`→Cars.`owner_id`.
- **Payments Table:** Similar to before, but when inserting a payment record, we also note the owner share. Can add fields `owner_amount` and `admin_amount`, or handle in business logic.
- **Payouts Table:** Logs money transferred to owners: (`payout_id`, `owner_id` FK, `amount`, `payout_date`, `status`). Admin uses this to track payments owed to owners (after commission).
- **Feedback/Ratings Table:** We expand feedback so that reviews can be about either a car or a user. For example: (`rating_id`, `booking_id`, `rating_by_user_id`, `rating_for_user_id`, `rating_value`, `comments`, `date`). This way, after a booking, a customer can rate the owner and vice versa. The blog suggests "owners and renters can provide ratings for each other" ⁸.
- **ChatMessages Table:** (`message_id`, `sender_id` FK, `recipient_id` FK, `content`, `timestamp`), unchanged conceptually, but now messages may flow between customers and owners as well.

Primary and foreign keys remain as before, with additional relations:

```
Users( user_id PK, ..., role )
Owners( owner_id PK, user_id FK → Users )
Cars( car_id PK, owner_id FK → Owners, ... )
Bookings( booking_id PK, car_id FK → Cars, user_id FK → Users, status, ... )
Payments( payment_id PK, booking_id FK → Bookings, amount, ... )
Payouts( payout_id PK, owner_id FK → Owners, amount, ... )
Ratings( rating_id PK, booking_id FK, rating_for_user_id FK → Users, ... )
```

```
ChatMessages( message_id PK, sender_id FK → Users, recipient_id FK → Users, ... )
```

Owners and customers both refer to `Users`, but owners have additional attributes.

Booking Workflow (Revised)

1. **Search & Request:** Customer searches and requests a booking on an owner's car. Backend creates a booking record with `status = PendingOwnerApproval`.
2. **Owner Approval:** The platform notifies the Owner (and optionally Admin). The Owner can **accept or decline** the booking ⁵. If accepted, the booking status becomes "OwnerApproved". If declined, it is cancelled.
3. **Payment:** Upon approval, the Customer is prompted to pay. The Customer completes payment through `POST /api/payments`. The system records the payment and sets booking status to "Confirmed". Payment processing is **securely handled** as in any rental app ¹².
4. **Pickup:** On the scheduled date, the Customer picks up the car. A controller endpoint (`/bookings/{id}/pickup`) marks the car as "In-Use" and updates status.
5. **Return & Billing:** After use, Customer returns the car. The Owner or Customer calls `/bookings/{id}/return`. The system then computes the final bill, including any extras (mileage, fuel, late fees).
6. **Payout:** Finally, the Admin processes payouts. The platform's commission is taken first, and the remaining amount is sent to the Owner ⁷. The Payouts table is updated. Customers receive a receipt of the transaction.

Throughout, messages can be exchanged: e.g., a chat may open when the booking is created, so **owners and renters coordinate** pick-up/drop-off times ⁶. Ratings are collected after completion, reinforcing trust between parties ⁸.

Data Flow Example

- A **Customer** on the React app calls `GET /api/cars?location=X&fromDate=...`. The Spring controller queries the `Cars` table (joined with `Owners` and `Bookings` to filter). JSON is returned to React.
- When booking, React posts to `POST /api/bookings` (customer token in header). The server's service layer verifies car availability and saves a new booking (status Pending).
- The Owner frontend polls or receives a notification. The Owner posts to `/api/owners/{ownerId}/bookings/{bookingId}/approve`. The Booking service checks that this owner owns the car, then sets status to Approved.
- Customer calls `POST /api/payments` with bookingId and payment info. The server charges via gateway, then on success records payment and updates booking status.

- Admin initiates a payout by calling `/api/admin/payouts` (or it happens automatically). The platform splits amounts and updates the `Payouts` table.
- Responses flow back through controllers to React components, updating the UI. This follows the Spring flow architecture (Controller→Service→Repository→DB) ² ³ .

References

We have incorporated best practices for a multi-sided rental platform. For example, peer-to-peer car rental solutions emphasize features like detailed owner listings, booking approvals, secure payments, and mutual ratings ⁴ ⁵ . The GitHub-based project explicitly describes the **three user roles** (owner, customer, admin) and the admin's role in verification and payment distribution ¹ ⁷ . Our design aligns with these principles: owners list and manage cars ⁴ , owners accept bookings ⁵ , and communication channels keep everyone coordinated ⁶ . All key points above are adapted from these sources to ensure a comprehensive, real-world approach to the expanded car rental system.

¹ ⁷ ⁹ ¹¹ GitHub - farazkhawaja/Car-rental-system: two-way car renting system(Semester Project) · GitHub

<https://github.com/farazkhawaja/Car-rental-system>

² Spring Boot - Architecture - GeeksforGeeks

<https://www.geeksforgeeks.org/springboot/spring-boot-architecture/>

³ reactjs - What are the architecture tiers of a Spring Boot + React + MySQL application? - Stack Overflow

<https://stackoverflow.com/questions/76542321/what-are-the-architecture-tiers-of-a-spring-boot-react-mysql-application>

⁴ ⁵ ⁶ ⁸ ¹² Blog - All Ride Apps

<https://www.allrideapps.com/peer-to-peer-car-rental-software-companies/>

¹⁰ How to make One to one chat application using spring boot and websockets - Stack Overflow

<https://stackoverflow.com/questions/48002128/how-to-make-one-to-one-chat-application-using-spring-boot-and-websockets>